Practice in Informatics
シェルスクリプト入門

山形大学
情報・エレクトロニクス学科
情報・知能コース

目次

1 **導入** 2
1.1 シェルの種類 3

2 **シェルスクリプトの基本構造** 4

3 **シェル変数** 5
3.1 特殊なシェル変数 6
3.2 整数演算 7

4 **if 文と条件文** 8
4.1 条件文 8
4.2 if 文 9

5 **実習操作その 1** 10

6 **ループ構造の制御** 11
6.1 for 文の構文 11
6.2 while 文 12

7 **有用な外部コマンド** 14
7.1 リダイレクトとパイプ 14
7.2 コマンド置換 15
7.3 sort による並びかえ 15
7.4 sed による置換 16
7.5 awk による欄表示 18

8 **実習操作その 2** 18

9 **レポートについて** 20

1 導入

シェル (shell) とは UNIX におけるプログラムの一つであり、主に OS に対するユーザインターフェースとして働く。一方、 言語としてはインタプリタ (interpreter) としてシェルスクリプト (shell script) に書かれた一連の命令を順に実行することができる。

この演習はシェルスクリプトを記述し、簡単なシェルを使ったプログラムを作成できるようになることを目標とする。プログラミング環境は通常の UNIX 環境を想定している。以下の章では順に読むことによってひととおりのシェルプログラミングが修得できるように書いているつもりである。順に**問題**を解いていって説明されている事項に対する経験を積みあげていってほしい。† のついた問題は演習では省略することがある。また**発展**はさらに知識を深めるためのものである。これらについては演習担当者の指示に従っていただきたい。

もとより完璧を期したものではなく使用頻度が低いと思われるものや最初の段階ではとばしてもよいと判断した事項は省略されている。興味ある方は文献などを参照して情報を補われたい。参考文献としては次のようなものが挙げられる。

参考文献

[1] オンラインマニュアル, `/usr/share/jman/man1/sh.1.gz`
書店で購入できるものとしては、

[2] 山口和紀ほか監修, 『**新** *The UNIX Super Text (***上***)(***改訂増補版***)*』 , 技術評論社, 2003. (旧版 1992).
が良書で、第 40 章に記述がある。この本には正規表現 (第 32 章)、sed と awk(第 35 章) についての記述もある。[*1]他の UNIX 入門のようなタイトルの本で、shell の説明に一章を割いている本も多く、そういったものでも十分に役に立つ。
また、ウェブ上にもたくさんみつけることができる.
次の本は平易で読みやすく、また手に入れやすいので初読には適していると思う。

[3] ブルース・プリン, 『**入門** *UNIX* **シェルプログラミング改訂第** *2* **版**』, ソフトバンク, 2003.
全体を詳しくかつ網羅的に知りたい場合は次の本を読むのがよい。

[4] Arnold Robbins, Nelson H. F. Beebe, 『**詳解シェルスクリプト**』, オライリー・ジャパン, 2006.
次も手頃なハンドブックである。

[5] 山森丈範, 『**シェルスクリプト基本リファレンス**』, 技術評論社, 2005.

[6] 大木敦雄, 『*UNIX* **シェル入門**』, 技術評論社, 2009.

[*1] 旧版ではそれぞれ第 30,25,27 章。

1.1 シェルの種類

通常、端末 (端末エミュレータ) を開いたとき目にする、ユーザの入力を促すために行の先頭に表示されているものをプロンプトという。"$", "%", ">" といった一文字の記号の事もあるし、”[user@eib-pc099:~>” のような場合もあるだろう。これは実はシェル (shell) というプログラムが後ろで動いており、対話的 (interactive) にユーザの入力を促しているものである。後ろで動いているシェルをログインシェル (login shell) という。

ログインシェルは (この教室のデフォルトでは) csh (シーシェル) または tcsh (ティーシーシェル) と呼ばれるものが使われている。

一方、非対話的 (non-interactive) に使用されるシェルとしては Bourne shell (ボーンシェル) または単に sh と呼ばれるものが一般的である。他にもこの高機能版である bash や zsh, ksh などがある。

この演習では Bourne shell 系のシェルを扱う。(今後単に sh と呼ぶ。) また、今後 sh を対話的に書く際、そのプロンプトを sh(sh,bash など) では $, login shell (tcsh など) では % と書くと約束する。

sh を対話的に使いたい場合、

```
% sh
$
```

と単に入力すればよい。プロンプトが変化するのでわかるだろう。sh から抜けるには

```
$ exit
%
```

とする。

オンラインマニュアルについて

このようなコマンドにはすべてにオンラインマニュアルが用意されている。全部読む必要はないが、ちょくちょく見て慣れたほうが上達が早いと思う。英語のマニュアルしか無い場合もあるが、眺めるだけで結構必要な事項はわかるものである。気にせず斜め読みでもよいから活用していただきたい。具体的には

% *man* **(コマンド名)**

と入力することにより指示したコマンドのオンラインマニュアルが表示される。例えば echo の場合、

% *man echo*

とすると次のような表示になる。

---

echo(1) echo(1)

名前

echo - 1 **行のテキストを表示する**

SYNOPSIS
書式

echo [-ne] [string...]
echo [--help] [--version]

説明

echo **は与えられた** string **をそれぞれ標準出力に書き出す。それぞれの間にはスペースを挟み、最後の引数の後には改行 を 行う。ただし** -n **オプションが指定されている場合は改行しない。（中略）**

オプション

-n　**行末の改行を行わない。**

-e　string **中の、バックスラッシュでエスケープされた文字の解釈を有効にする。（以下略）**

発展 1. 次のコマンドのオンラインマニュアルを開いてみよう。それを読んで、それぞれ何をするコマンドか理解しよう。

cd, cp, echo, ls, mv, pwd, wc

2　シェルスクリプトの基本構造

一連の動作を実現する目的でシェルのコマンドを書きならべて作ったテキストファイルをシェルスクリプト (shell script) という。シェルスクリプトを作成してみよう。

次のテキストファイルをエディタ (emacs[*2]) で入力してみる。

```
1:  #!/bin/sh
2:  # This is a Hello world program.
3:  echo "Hello, World."
4:  exit 0
5:  # end of hello.sh
```

これをファイル名 hello.sh として保存して、

```
% chmod +x hello.sh
```

[*2] ~/.emacs に (setq sh-shell-file "/bin/sh") と書いておく必要があるかもしれない。

を行い、実行可能にする。その後、

```
% ./hello.sh
```

とやると、

```
Hello, World.
```

と表示されるはずである。この一連の手順が、以降基本となるので繰り返し試されたい。

シェルスクリプトでは行単位で解釈されていく。一行目にある

```
1: #!/bin/sh
```

はログインシェルに対し、このファイルを /bin/sh で実行するように伝える文で、以降でもシェルスクリプトの一行目には必ず書く。また四行目の

```
4: exit 0
```

は省略可能だが明示的に終了地点を表わすことにより可読性があがることがあるのでここでは必ず書くことにする。

その他 # ではじまる行は コメント行と呼び、実行時には行末まで無視される。タイトルやわかりにくいところにはどんどんコメントをつけたほうがよい。

三行目の echo というのは 引数にくる文字列を表示する (外部) コマンドである。シェルスクリプトは複雑なプログラムを組むというよりはむしろ、いろいろな UNIX のコマンドをならべて目的を果たすのに使われる。

問題 2. 上記の hello.sh を実際に作って実行させてみよ。

3 シェル変数

シェルスクリプトにおいては変数を宣言することなくユーザが自由に定義できる。変数はアルファベットの大文字小文字、先頭にこない数字、及びアンダースコア (_) で構成される。なお予約語である if とかは変数として定義できない。変数への代入は**スペースをはさまない** = で定義されている。

```
$ FOO="test1.txt"        ← 変数 FOO に右辺の文字列を代入。
$ bar=2                  ← 変数 bar に文字列 2 を代入。
```

変数を評価/参照するには $ をその変数の先頭に置く。。後ろに別の文字が続くときは以下の例のように {} を使って区切る。評価を抑制したい場合は '$FOO' のようにシングルクォート (') で囲む。

表 1　特殊なシェル変数

$0	実行ファイル名 (argv[0])	$*n*	第 *n* 番目の引数 ($n = 1 \cdots 9$) (argv[*n*])
$*	引数リスト	$#	引数の数 (argc)
$$	プロセス番号	$@	引数リスト
$!	直前のバックグラウンドプロセス番号	$-	指定されているオプション

```
$ echo $FOO
test1.txt
$ echo $bar
2
$ echo ${FOO}_old
test1.txt_old
$ echo $FOO_old
```

← **変数** FOO_old **が定義されていないため何も表示されない。**

```
$ echo '$FOO'_old
$FOO_old
```

sh には予約変数がいくつかある. 主なものは HOME, PATH, SHELL などでこれらはシステムがあらかじめ設定を行う。その他、環境変数と呼ばれる変数もありシェル変数と同様に参照できる。[*3]環境変数は

```
% printenv
```

で一覧を見ることができる。

発展 3. HOME,PATH,SHELL, およびその他の環境変数にどのような値が入っているか表示させるスクリプトを書いてみよう。またマニュアル等文献をしらべてそれぞれどういう意味の変数か調べてみよう。

3.1　特殊なシェル変数

予約変数のほかにシェルには特殊な変数が存在する。これらは非常に良く使われるので表 1 にまとめておこう[*4]。これらは参照しかできず、常に $ と一緒で利用する。

シェルスクリプト自身のファイル名を $0 で参照できる。また、その引数リストおよび引数の数はそれぞれ $*, $# で参照することができる。例えば、

```
$ ./foo.sh arg1 arg2 arg3
```

[*3] シェル変数がそのシェルのみのローカルな変数なのに対し、環境変数はサブシェルにも渡すことのできる「グローバル」な変数である。

[*4] $*と$@は "$*" と"$@"のようにダブルクォートで囲んだときの挙動が異なる。

と実行したとすると、

$$\$0 \Rightarrow \boxed{\texttt{./foo.sh}},\quad \$* \Rightarrow \boxed{\texttt{arg1 arg2 arg3}},\quad \$\# \Rightarrow 3$$

が参照される。引数リストにアクセスするには、$のあとに一桁の数字 n を指定することで n 番目の引数を参照することができる。残念ながら 10 番目以降の引数は直接参照することはできない。次の shift というコマンドで、間接的にアクセスすることになる。

shift n	$\$i + n$ を $\$i$ に順にシフトする。$# はマイナス n される。なお shift 1 は単に shift と書ける。

問題 4. 右のシェルスクリプトを作成して、各自適当な引数を指定して実行してみよ。(シングルクォート'' で囲まれた部分は評価されずにそのまま表示されることに注意。)

```
1: #!/bin/sh
2: echo '$0'=$0, '$1'=$1, '$2'=$2.
3: echo '$#'=$#, '$*'=$*
4: shift
5: echo '$0'=$0, '$1'=$1, '$2'=$2.
6: echo '$#'=$#, '$*'=$*
7: exit 0
```

✎ **注意** ([2, 第 40.23.4 章]). これらの特殊変数は直接 = で代入することはできない。引数が 10 個未満であることがわかっていてどうしても数字部分を変数として引数リストにアクセスしたい場合は eval を使う方法があるといえばある。

問題 5. 結合コマンド cat.sh を作成せよ。これは

```
% ./cat.sh some thing
something
```

というふうに引数をつなぎあわせた結果を表示するコマンドである。

3.2 整数演算

残念ながら、シェルでは四則演算を

```
i = i + 1;    ### 間違い
```

のように単純に書くことはできない。かわりにシェル変数の算術展開を利用することになる。算術展開は、行いたい式を ((と)) で囲む。利用可能な演算子を表 2 にまとめておく。

算術展開のカッコ内の変数は $ を省略できるが、コマンドライン引数の場合など特殊なシェル変数の場合は、$(($1 + $2)) などと書く必要がある。算術展開の結果を参照する場合も先頭に$が必要になる。

表 2　算術展開における主な演算子

演算子	意味
`+ - * / %`	順に加算、減算、乗算、除算、剰余を行う。
`++ --`	変数の値を 1 加算または減算する。　(一部の sh の拡張機能)
`>> << & ^ |`	ビット演算。順に右シフト、左シフト、AND, XOR, OR を行う。(同)
`+= -= *= /= %=`	それぞれ対応する代入演算を行う。　(同)

```
$ x=3
$ echo $(( x + 2 ))
5
$ y=$(( x - 4 ))
$ echo $y
-1
```

問題 6. 足し算コマンド add.sh を作成しなさい。これは

```
% ./add.sh 8 12
20
```

というふうに引数を足しあわせ、答えを表示するコマンドである。

問題 7. 問題 6 を参考にして、それぞれ `- * / %` を用いて減算、掛算、割算、剰余算をするコマンド sub.sh、mul.sh、div.sh、 res.sh を作成しなさい。

問題 8. `shift` n と `$#` を使って最後の引数のみを表示するスクリプトを書きなさい。

4　if 文と条件文

4.1　条件文

shell での条件文は正常終了 (0) が真、 それ以外が偽として判定される。一見 C 言語などとは逆のようだが、正常終了なら真、と思うのが良い。

条件の一般的な書き方は次の二通りある。

- test 文を使う。

```
test   cond
```

- [ ] を使う。

```
[   cond   ]
```

表 4 test による条件文

文字列の比較	
test "$x" = "*string*"	変数 x の値と文字列 *string* が等しいと真。
test "$x" != "*string*"	変数 x の値と文字列 *string* が等しくないと真。
数字の比較	
test $x -gt *n*	変数 x の値が n より大きいと真。($x > n$)
test $x -lt *n*	変数 x の値が n より小さいと真。($x < n$)
test $x -ge *n*	変数 x の値が n 以上だと真。($x \geq n$)
test $x -le *n*	変数 x の値が n 以下だと真。($x \leq n$)
test $x -eq *n*	変数 x の値が n と等しいと真。($x == n$)
test $x -ne *n*	変数 x の値が n と等しくないと真。($x \neq n$)
ファイルの有無	
test -f *file*	*file* が存在し通常ファイルであれば真。
test -w *file*	*file* が存在し書きこみ可能な通常ファイルであれば真。
test -d *dir*	ディレクトリ *dir* が存在すれば真。
test -s *file*	*file* のサイズが 0 でなければ真。
test -r *file*	*file* が読みこみ可能であれば真。
test -x *file*	*file* が実行可能であれば真。

どちらも *cond* が正しいと真 (正常終了, 0 を返す) になり、正しくないと偽になる。また] はあくまで、[にとっての最終引数であるので、前後にスペースが必要である。具体的には条件文は例えば test を使って表 4 のようなものが挙げられる。なお、[test *cond*] のように混ぜて書かないように。

発展 9. test のオンラインマニュアルを調べて、(a) 単項否定演算子 (!), (b) 論理積演算子 (-a), (c) 論理和演算子 (-o) をみつけてそれぞれを使ってみよう。また ||, && とはどう違うだろうか。

4.2 if 文

条件分岐のため、 sh には if 文と case 文が用意されている。

```
if cond1 ; then list;  fi
if cond1 ; then list1; elif cond2 ; then list2 ;... else listN ; fi
```

if then fi は必須である。elif–then–else の部分は省略可である。また、一行で書くには ; (セミコロン) が必要なのに注意。改行を入れる場合にはセミコロンは必要ない。

条件文 *cond* が真 (0) だと、then 以下が実行され、そうでないと、else 以下が実行される。elif 文は else if を簡易にしたもの。*cond* が空になると文法エラーになる。*list* を空にしたいときには : (コロン) が使える。例えば、

```
if test $x -gt 0 ; then
    :
else
    （条件が偽のときに実行されるコマンド）
fi
```

のように else 文だけ、生かしたいときに使える。

問題 10. 問題 7,6 を参考にして演算コマンド calc.sh を作成せよ。これは

```
% ./calc.sh 8 '+' 12
20
% ./calc.sh 8 '*' 12
96
...
```

のように第二引数に '+', '-', '*', '/' を指定しその演算を行うコマンドである。

問題 11. `ls -l` で表示される先頭の 4 文字は, ディレクトリか (d), 読みこみ可能か (r), 書きこみ可能か (w), 実行可能か (x) をそれぞれ表している。これを `test` 文の `-d, -r, -w, -x` などを使ってシェルスクリプト `owner.sh` として実装してみよ。例えば次のようになる。

```
% ls -l
-rw-r--r--   1 hayata   staff          0 Jun 23 20:55 foo
-r-xr-xr-x   1 hayata   staff          0 Jun 23 20:55 owner.sh
% ./owner.sh foo
-rw-
% ./owner.sh owner.sh
-r-x
```

5　実習操作その 1

❒ **実習 1.** 問題 *2, 4, 5, 6, 7, 8, 10, 11* を解きなさい。

❒ **実習 2.** 次のシェルスクリプトを実行するとなんと表示されるかそれぞれ答えよ。またその理由を説明せよ。さらに *(1)* の *2* 行目を *ABC=$A; A=BC* に変更するとどうなるか説明せよ。

(1)

```
1:  #!/bin/sh
2:  ABC=A; A=BC
3:  echo A$ABC
4:  exit 0
```

(2)

```
1:  #!/bin/sh
2:  echo '$1' + '$2' = $(($1 + $2))
3:  echo "$1" + "$2" = $((1 + 2))
4:  exit 0
```

❒ **実習 3.** 次の *if* 文を含むシェルスクリプト

```
1:  #!/bin/sh
2:  x=abc; y=1+2; z=-2
3:  if [    (*)    ] ; then
4:  echo "true" ; fi
5:  exit 0
```

において、空欄 (*) には次の *(1)* から *(6)* の条件文がそれぞれ入っているとする。このとき実行すると *true* と表示されるものはどれか。またそれはどうしてか理由を説明せよ。*(*いくつかは文法に問題がある。*)*

(1) `test x$x = "xabc"`　　*(2)* `[ $xx = "abcx" ]`

(3) `test $y -eq 3`　　*(4)* `test 1+2 = y`

(5) `[ test $z = -2 ]`　　*(6)* `[ -$z = 2 ]`

❒ **実習 4.**

次のシェルスクリプト *j4.sh* を以下のように実行した際、表示される文字列をそれぞれ答えよ。またその理由を答えよ。

(1). `% ./j4.sh 13`
(2). `% ./j4.sh 10`
(3). `% ./j4.sh 2`

./j4.sh:

```
1:  #!/bin/sh
2:  if test $1 -lt 10 ; then
3:          i=$(($1 + 1))
4:          $0 $i
5:  else
6:          echo $1
7:  fi
8:  exit 0
```

6　ループ構造の制御

6.1　for 文の構文

sh においても繰り返しのコマンドがいくつか用意されていて、その一つが for 文である。for 文の文法は

for *name* in *list* ; do *prog* ; done

ここで、for, do 及び done は必須である。*name* が *list* の要素をひとつずつ動き、ひとつにつき 一回ずつ do と done のあいだを実行する。

例えば次のプログラムは a, b, c を順に表示する。

```
1:  #!/bin/sh
2:  for x in a b c
3:  do
4:      echo $x
5:  done
6:  exit 0
```

このように各ブロックを改行で区切れば、; (セミコロン) は必要ない。逆に複数行を一行に書くこともできるが、その際は ; をはさむ必要がある。

問題 12.
右のスクリプトを参考にして、ディレクトリ/etc 内にあるファイルまたはディレクトリの数を数え、表示するプログラムを作成しなさい。さらに、

(1). /etc にある p ではじまるファイルもしくはディレクトリ、
(2). /etc にある .conf で終わるファイル

を数えるスクリプトに書きかえてみなさい。

./q12.sh:

```
        ......
for x in /etc/*
do
    i=$((i + 1))
done
        ......
### * はワイルドカード。
```

6.2 while 文

while *cond* ; do *list* ; done

for 文と同様に do と done の間をループする。ただし、*cond* を実行した結果が 真 (0) の場合、do *list* を行なう。そうでない場合、ループは終了し、done の次に処理がうつる。

問題 13. 次のシェルスクリプトは何をするプログラムか。

```
1:  #!/bin/sh
2:  sum=0
3:  j=0
4:  while [ $j -lt 10 ]
5:  do
6:      sum=$(( sum + j ))
7:          j=$(( j + 1 ))
8:          echo $sum
```

```
done
exit 0
```

問題 14. 問題 13 のシェルスクリプトにおいて、while 文をすべて一行に書きなおしてみよ。(do と done の間に複数行あることに注意。)

発展 15. †while 文とよく似たループに until 文というのがある。マニュアル等文献を調べ、(a) どのような文法で、(b) どのように使うのか、を調べてみよう。

問題 16. 問題 5 の cat.sh を

```
% ./cat.sh int erna tion aliza tion
internationalization
```

というふうに幾つ引数がきてもすべてつなぎあわすことができるように修正せよ。

問題 17. 問題 6 の add.sh を

```
% ./add.sh 8 12 2 3
25
```

というふうに幾つ引数がきてもすべて足した数が表示されるように修正せよ。

なお、while true; do ... ; done で、無限ループを作ることができる。またループからは break 文を使って脱出することができる。また、シェルにおいては変数が局所的でないのに注意されたい。ループから break 文で脱出した際など、ループのランニングインデックスを (ループの外で) 利用するのは避けたほうがよい。

問題 18. †case 文は pattern が決まっているときの条件分岐で、次の文法をもつ。

case *word* in *pattern*) *list* ;; esac

次の例はコマンドオプションをひとつずつチェックして変数 OPTIONS に追加する シェルスクリプトである。

```
#!/bin/sh
OPTIONS=""
while true
do
        case "$1" in
        -?*) OPTIONS="$OPTIONS $1" ;;
        *)  break ;;
        esac
        shift
```

```
10:  done
11:  echo $OPTIONS
12:  exit 0
```

(1). 上のシェルスクリプトを入力し実行してみよ。
(2). 上のシェルスクリプトを `if` 文を使って書きなおせ。

7　有用な外部コマンド

シェルスクリプトは sh のコマンドだけでなく、いろいろな (外部) コマンドを利用することによって様々な処理を実現するほうがむしろ普通である。たくさんある外部コマンドの内、ここでは awk, sed 及び sort を使ってみることにしよう。その他、grep, find など有用なコマンドが数多くある。各自調べてみてほしい。

7.1　リダイレクトとパイプ

unix 流のテキスト処理では複雑な作業を、単純な外部コマンドを複数つなげることで実現する。まず標準入力、標準出力、標準エラー出力の違いを理解し、これらの指定の仕方を覚え、自由につなぎあわせられるようになろう。

< > >> を**リダイレクト (redirect)**、| を**パイプ (pipe)** 主な使い方を表 6 に挙げた。[*5]

リダイレクトは複数指定することができる。複数指定されたときは、左から順に割当てが決まっていく。ただしパイプは例外で、すべてのリダイレクトの評価の前にまず、パイプが評価される。

問題 19. † 次のリダイレクト

$ コマンド > *file* 2>&1
$ コマンド 2>&1 > *file*
$ コマンド *A* 2>&1 1>&- |コマンド *B*
$ コマンド *A* 1>&- 2>&1 |コマンド *B*

は同じだろうか。それぞれの挙動を説明せよ。

発展 20. †<< をヒアドキュメントという。これはどのようなリダイレクトかを調べてみよう。さらにそれを使ったスクリプトを作ってみよう。

[*5] *m*>&*n*, *m*<&*n* で *m* を「*n* と同じものをわりあてる」の意になる。*m*, *n* は 0 ~ 9 が指定できる。

表 6　リダイレクトの主な使い方

例	意味
コマンド < *file*	標準入力として *file* を読み込み用に開く。
コマンド > *file*	標準出力として *file* を書きこみ用に開く。
コマンド 1> *file*	(> と同じ。)
コマンド >> *file*	標準出力として (すでにある)*file* を追加書きこみ用に開く。
コマンド 2> *file*	標準エラー出力 として *file* を書きこみ用に開く。
2>&1	標準エラー出力として標準出力と同じものを書きこみ用に開く。
1>&-	標準出力を閉じる。
コマンド *A* | コマンド *B*	コマンド A の標準出力をコマンド B の標準入力につなぐ。
コマンド *A* 2>&1 | コマンド *B*	コマンド A の標準出力と標準エラー出力をコマンド B の標準入力につなぐ。
コマンド *A* 2>&1 1>&- | コマンド *B*	コマンド A の標準エラー出力 (のみ) をコマンド B の標準入力につなぐ。

7.2　コマンド置換

外部コマンドが実行された後、表示される標準出力の内容を変数に保存したい場合、**コマンド置換**を用いる。コマンド置換はバッククォート (`) でコマンドを囲むことで実行される。`で囲まれたコマンドは出力がそのまま評価結果に置きかわる。

```
$ echo abc
abc
$ x=` echo abc`← 何も表示されない。
$ echo $x
abc
```

7.3　sort による並びかえ

sort コマンドは入力ファイルを行単位で並びかえる。オプションをいろいろ設定することでいろいろな並びかえを実現できる。基本的な使い方は以下のとおり。

sort [**オプション**] **入力ファイル**

入力ファイルが省略されると標準入力からの入力になる。

次の例は /etc/group を並びかえている。

```
% sort /etc/group
adm:x:4:root,adm,daemon
```

表 8　sort のオプション

オプション	意味
-n	数字として比較し並びかえる。
-r	逆順に並びかえる。
-t *fs*	*fs* を欄の区別子とみなす。(デフォルトの区別子は空白文字。)
-k *pos1*,*pos2*	第 *pos1* 欄から第 *pos2* 欄をキーとして並びかえる (*pos2* は省略可)。
-u	同じ行があったらその先頭の行のみを表示する。

```
apache:x:48:
avahi:x:70:
bin:x:1:root,bin,daemon
daemon:x:2:root,bin,daemon
(中略)
wheel:x:10:root
xfs:x:43:
```

ちゃんとアルファベット順になっていることがわかる。

sort には有用なオプションが用意されている。いくつかを表 8 に載せる。

問題 21. 次を実行してみよ。そしてそれぞれのオプションがどういう役割を果たしたかを述べよ。

```
% sort -r -t : -n -k 3 /etc/group
```

7.4　sed による置換

sed も昔からよく使われてきた多機能な行指向編集コマンドである。ここでは sed を使った文字列の置換を紹介する。

```
sed -e 's/正規表現/foo/' 入力ファイル
sed -e 's/正規表現/foo/g' 入力ファイル
```

正規表現に最初にマッチした文字列が *foo* に置き換わる。マッチの仕方が複数ある場合は「左優先」になる。すなわち左にある正規表現から順に長くなるようにマッチが行われる。

g をつけるとマッチしたすべてを置換する。入力ファイルを省略すると標準入力からの入力となる。また s コマンドが一つしか指定されないときは-e は省略できる。ちなみに上では区切文字を / としたが、実際はなんでもよい。以下のように次に来る一文字が区切文字と認識される。

```
% echo "foo bar foo1" | sed 's/foo/hoge/'
hoge bar foo1
```

表 9　sed の主な正規表現

トークン	意味
^	文頭を表す。
$	文末をあらわす。
.	任意の一文字を表す。
p_1*	正規表現 p_1 の 0 回以上の繰り返しを表す。
[$c_1c_2\cdots c_n$]	c_1 から c_n までの文字のうちの一文字を表す。先頭に],-,^がくるときを除き、. 等の特殊文字もそれ自身の意になる。
(例)　[abc]	・a または b または c のどれか一文字。
[a-z]	・文字のアルファベットのうちどれか一文字。
[0-9]*	・数字の 0 文字以上の繰り返し。
[^$c_1c_2\cdots c_n$]	括弧内の先頭にキャラット (^) をつけると、[] 内以外の一文字を表す。
(例)　[^0-9a-zA-Z]*	・数字およびアルファベット以外の文字の 0 文字以上の繰り返し。

```
% echo "foo bar foo1" | sed 's:foo:hoge:g'
hoge bar hoge1
% echo "foo1 bar2 baz3" | sed  's/\([a-z]*\)[0-9]*/\1/g'
foo bar baz
% echo "foo1 bar2 baz3" | sed 's/[a-z]*\([0-9]*\)/foo\1/g'
foo1 foo2 foo3
```

ただし \1 は \(と\) で囲まれた部分を参照する。

問題 22. 次のコマンドを実行するとどういう文字列が出力されるか。

(1). `echo "<html><body>" |sed  's/<\([^>]*\)>.*/\1/'`
(2). `echo "int main (void)" |sed 's:(\([a-z]*\)):\1:'`
(3). `echo "int main (void)" |sed 's:.*(\([a-z]*\)):\1:'`
(4). `echo "/tmp/mytest.sh.tgz" |sed 's![^.]*\(\..*\)!\1!'`
(5). `echo "/tmp/mytest.sh.tgz" |sed 's=[/.]=-=g'`

問題 23. あるディレクトリには次のような TEX ファイルがある。

```
% ls
bar1.tex   bar2.tex   bar4.tex   bar99.tex   foo4.tex   foo5.tex
```

ところが、bar をファイル名に含むファイルにおいて \begin{...} と書かなければいけないところを間違って \bgein{...} と書いてしまった。これらを修正するシェルスクリプトを for ループを利用して作りなさい。(ヒント： 例えば、mv でファイル名を変えて一時ファイルを作り、sed で処理したのち、rm で削除するとよい。)

7.5 awk による欄表示

awk ’ **パターン** { **スクリプト** }’ **入力ファイル名**

awk はデフォルトではスペースまたはタブを区切文字として各行を欄に分けている。*i* 番目の欄は $*i* であらわされる。$0 で行全体、$NF で最後の欄をあらわす。表示は {print $*i*} または C 言語のように{printf("%s\n",$*i*)} などで行うことができる。また四則演算も C 言語と同じようにできる。

awk の区切り文字は、

```
% awk -F fs ...
```

のように -F オプションで指定できる。次のスクリプトは/etc/group の先頭のグループ名とそのグループ ID を表示している。

```
% awk -F : '{print $1":"$3}' /etc/group
root:0
bin:1
daemon:2
sys:3
...(中略)
apache:48
```

問題 24. ファイル /etc/group に対して、

```
% ./getgid.sh /etc/group
500=postfix
102=postgres
101=postdrop
100=users
99=nobody
  ... (省略)
2=daemon
1=bin
0=root
```

と，「グループ ID = グループ名」の形で降順に表示されるようにシェルスクリプト getgid.sh を作りなさい。

8 実習操作その 2

❒ **実習 5.** 問題 *12, 13, 14, 16, 17, 21, 22, 23, 24* を解きなさい。

❒ **実習 6.** 正の整数の入力 n に対し、n 以下の正の奇数の和を求めるシェルスクリプト *j6.sh* を作りなさい。

❒ **実習 7.** カレントディレクトリに

「ex + 数字」

を名に持つ通常ファイルおよびディレクトリがいくつかある。そのうち通常ファイルの数字部分を 1 増やすシェルスクリプト *(j7.sh とする)* を作成せよ。実行結果は例えば、次のようになる。*(なお、ls -F はディレクトリを /, 実行可能ファイルを* で示す。*)*

```
% ls -F
ex1    ex3     ex5/    ex7/    ex101    ex103/     j7.sh*
% ./j7.sh
% ls -F
ex2    ex4     ex5/    ex7/    ex102    ex103/     j7.sh*
```

なお、次のスクリプトを参考にしてもよい。

```
1:  #!/bin/sh
2:  for x in ex*; do n=`echo $x|sed 's/ex\([0-9][0-9]*\)/\1/'`;done;exit 0
```

❒ **実習 8.** あるディレクトリには次のような *4* つのファイル *(foo?)* と *4* つのディレクトリ *(dir?)* がある。

```
% ls -F
dir1/  dir2/  dir3/  dir4/  foo1   foo2   foo3   foo4
```

(1). すべてのファイルおよびディレクトリに *.org* という拡張子をつけるシェルスクリプトを作れ。
(2). 逆に *foo1.org* → *foo1* のように拡張子 *.org* を取りさるシェルスクリプトを作れ。
(3). ディレクトリだけに *.new* という拡張子をつけるシェルスクリプトを作れ。
(4). すべてのファイルに対し、引数に指定した文字列を拡張子としてつけるシェルスクリプトを作れ。

❒ **実習 9.** *UNIX* における */etc/services* は次のフォーマット

サービス名　ポート番号/プロトコル名　[サービス別名]

を持つテキストファイルである。これらに対し、

サービス名 (ポート番号, プロトコル名)

の形でアルファベット順に表示するシェルスクリプト *j9.sh* を作成せよ。

9 レポートについて

- この実習の評価には出席とレポート提出（受理）が必要である。
- 原則として当テキストの実習操作その 1（実習 1 〜 4）の全部と、実習操作その 2（実習 5 〜 9）から、一部または全部を選んでレポート作成する。
- スクリプト作成の問題では、**単にスクリプトを載せるだけではなく**、スクリプトの目的、実行例を用いた検証および動作原理の説明を、自分の言葉で行っていただきたい。
- レポートには、用意されたテンプレートを用いて、実習日、学生番号、氏名を必ず記載すること。
- 出力の表示が数ページにわたるなど冗長にならないように適宜気をつけること。また右端が切れて判読できないことのないよう気をつけること。
- 提出の方法、時期などの詳細は演習担当者の指示に従うこと。

$Id: bsh.tex,v 1.26 2009/08/23 15:35:25 hayata Exp $

2009 執筆担当: 早田孝博.